

数据库课程项目计划书

MusicQuery Agent——基于自然语言交互的
音乐作品库智能查询系统

团队名称：旋律引擎团队

团队成员：【待填写】

指导教师：【待填写】

编制日期：2026 年 9 月

一、项目概述

在音乐产业数字化进程中，独立音乐人工作室、中小厂牌、音乐培训机构和校园音乐社团等中小机构将作品、用户与播放数据沉淀在 MySQL 等关系型数据库中，却因缺乏 SQL 技能与专职数据人员而无法有效利用这些数据；通用 BI 工具价格高、门槛高，通用大模型产品直连数据库又存在安全风险。

本项目提出并实现 MusicQuery Agent：以自然语言为入口、以数据库设计为核心、以 Agent 为交互层的智能查询系统。系统将用户提问拆解为意图判断、SQL 生成、执行校验、结果解释四步流水线，配套三道安全防线与失败自修复机制，使不懂 SQL 的业务人员能够安全、准确地以中文查询数据库。

项目目标：（1）交付可运行、可复现、已开源的完整系统；（2）所有数据库设计（6 表、4 索引）有建模论证与 EXPLAIN 实证；（3）形成可复用的“AI 辅助编码 + 人工验收”团队协作规范。项目意义在于打通中小音乐机构数据价值变现的最后一公里，并为团队成员建立完整的大模型应用开发能力。

二、需求分析

2.1 功能需求

编号	需求	说明
F1	自然语言数据问答	中文提问，返回 SQL、结果表格与 50 字以内自然语言结论
F2	意图守门	非音乐业务问题自动识别并礼貌拒绝
F3	SQL 生成	依据真实表结构（DDL 注入提示词）生成只读 SELECT 语句
F4	安全校验与拦截	SELECT 白名单 + 危险关键词黑名单 + 被拦截请求留痕
F5	失败自修复	执行报错回传大模型修正，最多重试 2 次
F6	查询日志	问题、SQL、成功标志、耗时写入 query_logs 表
F7	历史记录展示	侧边栏展示最近 10 条查询及

		状态
F8	测试数据生成	一键生成 50/100/500/200/5000 规模、 可复现的仿真数据

2.2 非功能需求

安全性：任何情况下不得执行修改数据的语句，敏感请求须可审计；准确性：生成 SQL 必须严格对应真实表结构，数字结论必须来自查询结果本身；易用性：业务人员零学习成本，像聊天一样提问；可维护性：表结构演进时提示词零维护（运行时读取 schema.sql）；成本可控：优先使用免费档模型接口。

三、总体设计

3.1 系统架构

系统采用四步流水线架构：用户在 Streamlit 网页输入中文问题后，依次经过意图判断（大模型输出 YES/NO 守门）、SQL 生成（DDL 与字段说明注入提示词）、执行校验（安全拦截 + 执行 + 报错回传自修复循环，最多重试 2 次）、结果解释（查询结果转 JSON 后生成 50 字结论）；每次查询写入 query_logs 日志表，页面下方按“SQL / 结果表格 / 自然语言回答”三块展示，侧边栏呈现运行状态与历史记录。

3.2 技术选型

层	选型	理由
数据库	MySQL 8.4	课程核心；InnoDB 支持外键与事务；utf8mb4 支持中文
驱动	PyMySQL + DictCursor	轻量、参数化查询防注入、结果按列名取用
界面	Streamlit	Python 生态内快速搭建 Web 界面
大模型	智谱 GLM（OpenAI 兼容接口）	免费档可用；切换厂商仅需改 base_url 与模型名
配置	python-dotenv	密钥与配置隔离于 .env，不进入代码仓库
版本管理	Git / GitHub	里程碑打标签，公开仓库形成

		作品集
--	--	-----

四、数据库设计

4.1 表结构

表	角色	关键字段
artists	维度表：歌手	artist_id PK、name、country、debut_year
albums	维度表：专辑	album_id PK、artist_id FK、release_date、album_type (ENUM)
songs	维度表：歌曲	song_id PK、album_id FK、artist_id FK、duration_sec、genre、release_year
users	维度表：用户	user_id PK、nickname、city、register_date
play_records	事实表：播放记录	record_id BIGINT PK、song_id FK、user_id FK、play_time、device
query_logs	Agent 日志表	user_question、generated_sql、is_success、exec_time_ms、created_at

4.2 设计要点

（1）规范化：各表满足三范式，非主属性完全、直接依赖主键；唯一一处受控冗余是 songs 同时持有 album_id 与 artist_id，用于避免按歌手查歌时的二次 JOIN，一致性由导入层保证并以外键约束兜底。（2）完整性：全部 InnoDB 引擎 + 外键约束，album_type 使用 ENUM 域约束。（3）索引：songs(artist_id)、songs(genre, release_year)、play_records(play_time)、play_records(song_id, play_time) 四个索引分别服务按歌手查歌、流派年份组合筛选、时间范围过滤、单曲播放统计四类高频查询；复合索引顺序遵循最左前缀原则。（4）事实表主键采用 BIGINT 为规模化预留空间。

五、功能模块设计

意图判断模块：以极简提示词约束模型仅输出 YES/NO，无关问题一步拦截并返回“该问题与音乐数据库无关”。

SQL 生成模块：将 schema.sql 全文与字段业务说明、严格规则（只允许 SELECT、字段必须来自 DDL、列表结果加 LIMIT）注入提示词，温度参数取 0；对输出做代码块剥离与首条语句截取。

执行校验模块：先做语句白名单（仅放行 SELECT/WITH 开头）与危险关键词正则黑名单（DROP、DELETE、UPDATE、INSERT、ALTER、TRUNCATE 等）双重校验，再执行；执行失败时将报错原文回传大模型修正，修正结果重新过安全校验，最多重试 2 次，每次尝试均写入 query_logs。

结果解释模块：将查询结果（截取前 15 条）转为 JSON 传入提示词，生成不超过 50 字、不提及 SQL 与数据库字眼的中文结论；解释失败时退化为固定话术，不影响查询本身。

安全与容错总结：三道防线（白名单、黑名单、日志留痕）构成纵向防御；报错回传自修复构成横向容错闭环。被拦截请求同样留痕，实现全程可审计。

六、数据方案

测试数据使用 Faker 按任务书规定规模生成：50 歌手、100 专辑、500 歌曲、200 用户、5000 播放记录，按外键顺序批量插入；固定随机种子保证任何环境重建完全一致的数据集；脚本可重复执行（先清空业务表再导入，整体事务提交保证原子性）。

分布设计：播放记录采用对数正态分布模拟真实热度长尾——头部歌曲播放量近 200 次、长尾歌曲个位数；用户活跃度同样采用长尾。相比均匀随机（前五名并列、

无区分度），该设计使“播放量 TOP-N”等演示场景具备业务说服力。对比实验表明：均匀分布下前五名为 20/19/19/19/19，长尾分布下为 194/160/130/112/106。

七、实施计划与分工

阶段	任务	产出	负责人
第 1 周	选题讨论、需求分析、技术路线调研	需求清单、技术选型结论	全体（组长主持讨论会）
第 2 周	数据库建模、schema.sql 编写、环境搭建	6 表建库脚本、连接层	组长
第 3 周	Agent 四步流程开发、提示词设计	可运行系统 v0.9	组长（AI 辅助编码，人工验收）
第 4 周	安全防线、失败自修复、日志留痕	v1.0 本地存档（Git 标签）	组长
第 5 周	Streamlit 界面、测试数据方案、功能测试	v1.1 公开发布	队员
第 6 周	答辩材料：PPT、讲稿、详案，彩排	答辩包	全体

协作机制：每周例会同步进展并对分歧表决；代码经 Git 分支开发、合并前人工验收；“AI 产出、人工验收”规范约束全部自动化产出——架构决策、安全方案与数据库建模必须由成员讨论拍板，AI 生成内容须经运行测试、改动实验或 EXPLAIN 实证方可合入。

八、测试方案

用例	输入	预期	结果
T1 正常查询	播放量最高的前 5 首歌是哪些？	三表 JOIN 排序 SQL，返回 5 行，自然语言结论	通过
T2 聚合查询	一共有多少首流行歌曲？	COUNT 聚合 SQL，数字准确	通过
T3 意图拦截	今天天气怎么样？	提示“该问题与音乐数据库无关”	通过
T4 安全拦截	删除所有播放记录	DELETE 命中黑名单被拒，日志 is_success=0	通过

T5 失败自修复	构造易错问句	报错回传修正后重试成功, attempts>1	通过
T6 索引实证	三条代表性查询 EXPLAIN	分别命中 idx_songs_genre_year / idx_play_song_time / idx_play_time, 覆盖索引免回表	通过
T7 数据可复现	重复运行 import_data.py	各表行数 50/100/500/200/5000 完全一致	通过

性能验证：EXPLAIN 显示代表性查询均为 range 级别，扫描行数在数百行以内；
song_id+时间查询以覆盖索引执行（Using index），无需回表。

九、风险分析与应对

风险	影响	应对
大模型接口不可用/限流	无法生成 SQL	OpenAI 兼容接口可切换多家厂商（改 base_url 即可）； 报错自修复降低对模型质量的依赖
模型生成 SQL 质量不稳	查询失败	三道防线兜底 + 报错回传修正 + 日志留痕分析 badcase
数据库故障	服务不可用	错误日志落盘、启动检查清单、服务化部署（可注册 Windows 服务）
密钥泄露	账户损失	.env 不入仓库、仓库公开前全历史密钥扫描
免费模型政策变化	成本上升	模型切换零代码改动；成本结构轻，波动可承受

十、预期成果

（1）可运行、可复现、已开源的完整系统（GitHub 公开仓库，版本标签 v1.0/v1.1）；（2）完整数据库设计文档：6 表建模、三范式论证、4 索引设计与 EXPLAIN 实证；（3）规范的测试记录：7 项功能用例全部通过；（4）答辩材料包：10 页演示文稿（含逐页讲稿）、答辩详案、演示脚本；（5）团队级方法论沉淀：“AI 辅助编码 + 人工验收”协作规范与三次讨论记录。

十一、AI 工具与团队协作说明

本项目严格按照“AI + 小组讨论相结合”的方式推进。AI 的角色定位为“虚拟第三成员”，承担初版代码生成、技术调研与文案撰写；小组的角色是决策与验收：选题、技术路线、数据库建模、安全方案均经正式讨论表决（选题会、技术选型会、数据与安全设计会共三次，均有记录）。

验收机制包括三类实验：运行测试（7 项功能用例）、改动实验（修改提示词、重试次数、数据分布等参数观察系统行为变化）、实证检查（EXPLAIN 验证索引、日志验证拦截记录）。全部 AI 产出经上述验收后方可合入代码仓库。

该协作方式的效果已由可核查的证据支撑：GitHub 公开仓库的提交历史、v1.0/v1.1 版本标签、测试数据重建的可复现性、EXPLAIN 实测结果。团队认为，“AI 提效、人类把关”正是大模型时代软件工程的应有范式。